API-documentatie

Restaurantreserveringen, bestellingen en menukaart

Ali Khoshkam

Leerlijn Backend (30 EC) – Eindopdracht versie 4.3

Inleverdatum: 8 november 2026

Documentversie: 1.1 | LaTeX-bron 8 november 2026

Inhoudsopgave

1 Gebruik, authenticatie en foutcodes 2

2 Categorieën 2

3 Producten 2

4 Tafels 2

5 Reserveringen 3

6 Bestellingen 3

7 Facturen 3

8 Validatie per invoertype 4

9 Afbeeldingen en autorisatie 4

1 Gebruik, authenticatie en foutcodes

Dit is de API van het restaurant. Lokaal draait hij op `http://localhost:8080`. Importeer `exports/restaurant.postman_collection.json` en loop de requests per map af.

Voor gewone logins en registratie: Authorization Code + PKCE (S256) via Keycloak op `http://localhost:8081/realms/restaurant`. Client: `restaurant-postman`; callback: `https://oauth.pstm` Scope: `openid profile email`.

Lokale demoaccounts: `klant` / `Klant-Demo-2026!` (CUSTOMER) en `personeel` / `Personeel-Demo-2026!` (STAFF). Nieuwe registraties krijgen CUSTOMER. Dit zijn uitsluitend lokale demogegevens.

```
Authorization: Bearer <access_token>
Content-Type: application/json
```

GET geeft 200; POST 201 met Location; PUT 200 (afbeelding 204); DELETE 204. Fouten: 400 invoer, 401 token, 403 rechten/eigenaarschap, 404 ontbrekend, 405 methode, 409 conflict, 413 uploadgrootte, 415 bestandstype. Applicatiefouten gebruiken ProblemDetail.

```
{"type":"about:blank","title":"Conflict","status":409,
 "detail":"Table is reserved during this time slot"}
```

Ids zijn positieve database-identifiers. Datum/tijd is UTC zonder suffix; bedragen zijn EUR-eindprijzen. Er zijn geen lijstfilters of paginering.

2 Categorieën

Categorie beheren. Verwijderen is alleen mogelijk zonder gekoppelde producten. GET: CUSTOMER en STAFF. POST, PUT, DELETE: STAFF.

`GET /api/categories` — lijst (200). `GET /api/categories/{id}` — één object (200/404). `POST /api/categories` — aanmaken (201 + Location). `PUT /api/categories/{id}` — wijzigen (200). `DELETE /api/categories/{id}` — verwijderen indien ongebruikt (204/409).

```
POST {"name":"Desserts","description":"Nagerechten"}
201  {"id":3,"name":"Desserts","description":"Nagerechten"}
```

3 Producten

Producten beheren. Een gebruikt product kan niet worden verwijderd; zet `available` op `false`. GET: CUSTOMER en STAFF. POST, PUT, DELETE: STAFF.

`GET /api/products` — lijst (200). `GET /api/products/{id}` — één product (200/404). `POST /api/products` — aanmaken (201 + Location). `PUT /api/products/{id}` — wijzigen (200). `DELETE /api/products/{id}` — verwijderen indien toegestaan (204/409).

```
POST {"name":"Tiramisu","categoryId":1,"price":7.5,"available":true}
201  {"id":3,"name":"Tiramisu","categoryId":1,"price":7.5,
      "available":true,"imageUrl":"/api/products/3/image"}
```

4 Tafels

Tafels beheren. `available` geeft aan of een tafel beschikbaar is voor gebruik. Reserveringen bepalen of een tafel op een bepaald tijdstip bezet is. GET: CUSTOMER en STAFF. POST, PUT, DELETE: STAFF.

`GET /api/tables` — lijst (200). `GET /api/tables/{id}` — één tafel (200/404). `POST /api/tables` — aanmaken (201 + Location). `PUT /api/tables/{id}` — wijzigen (200). `DELETE`

`/api/tables/{id}` — verwijderen indien toegestaan (204/409).

```
POST {"tableNumber":3,"seats":4,"available":true}
201  {"id":3,"tableNumber":3,"seats":4,"available":true}
```

5 Reserveringen

Klanten beheren eigen reserveringen; personeel mag alle reserveringen lezen, wijzigen en annuleren. DELETE is een soft delete (annuleren). Duur: twee uur.

`GET /api/reservations` — lijst (200). `GET /api/reservations/{id}` — één reservering (200/404). `POST /api/reservations` — aanmaken (201 + Location). `PUT /api/reservations/{id}` — wijzigen (200). `DELETE /api/reservations/{id}` — annuleren (204).

```
POST {"customerName":"Ali","tableId":1,
      "startsAt":"2030-12-01T17:00:00","guests":2,
      "remarks":"Bij het raam"}
201  {"id":2,"customerName":"Ali","tableId":1,
      "startsAt":"2030-12-01T17:00:00",
      "endsAt":"2030-12-01T19:00:00","guests":2,
      "remarks":"Bij het raam","cancelled":false}
```

6 Bestellingen

Bestellingen beheren. Uitsluitend STAFF. Prijzen en productnamen worden opgeslagen op het moment van bestellen. De server berekent het totaalbedrag. Gefactureerde bestellingen kunnen niet meer worden gewijzigd.

`GET /api/orders` — lijst (200). `GET /api/orders/{id}` — één bestelling (200/404). `POST /api/orders` — aanmaken (201 + Location). `PUT /api/orders/{id}` — wijzigen (200). `DELETE /api/orders/{id}` — verwijderen indien toegestaan (204/409).

```
POST {"tableId":1,"lines":[{"productId":1,"quantity":2}]}
201  {"id":1,"tableId":1,"createdAt":"2030-12-01T17:10:00",
      "lines":[{"productId":1,"productName":"Pasta",
                "unitPrice":14.5,"quantity":2}],
      "total":29.0}
```

7 Facturen

Facturen beheren. Uitsluitend STAFF. Per bestelling kan maximaal één factuur worden aangemaakt. Met PUT kan alleen de betaalstatus (`paid`) worden gewijzigd. Betaalde facturen kunnen niet worden verwijderd of teruggezet naar onbetaald.

`GET /api/invoices` — lijst (200). `GET /api/invoices/{id}` — één factuur (200/404). `POST /api/invoices` — aanmaken (201 + Location). `PUT /api/invoices/{id}` — betaalstatus wijzigen (200). `DELETE /api/invoices/{id}` — onbetaalde factuur verwijderen (204/409).

```
POST {"orderId":1}
201  {"id":1,"orderId":1,"createdAt":"2030-12-01T18:00:00",
      "total":29.0,"paid":false}
PUT  {"paid":true}
```

8 Validatie per invoertype

Categorie `name` verplicht, niet leeg, max. 80 tekens, uniek; `description` verplicht, max. 500 tekens.

Product `name` max. 100 tekens; `categoryId` bestaand; `price` niet negatief, max. acht gehele en twee decimale cijfers; `available` boolean.

Tafel `tableNumber` ≥ 1 en uniek; `seats` 1–50; capaciteitsverlaging geweigerd zolang reserveringen gekoppeld zijn.

Reservering `customerName` max. 100 tekens; `startsAt` in de toekomst; `guests` 1–50 en passend bij de tafel; `remarks` max. 1000 tekens. Subject komt uit het token.

Bestelling `lines` 1–100; product uniek en beschikbaar; `quantity` 1–100. `total` en `createdAt` zijn servervelden.

Factuur POST: alleen `orderId` van een nog niet gefactureerde bestelling. PUT: boolean `paid`.

9 Afbeeldingen en autorisatie

`PUT /api/products/{productId}/image` — STAFF, multipart veld `file`, succes 204. Alleen PNG/JPEG tot 2 MiB en 16 miljoen pixels.

```
curl.exe -X PUT http://localhost:8080/api/products/1/image ^
  -H "Authorization: Bearer TOKEN" -F "file=@product.png"
```

`GET /api/products/{productId}/image` — CUSTOMER en STAFF, bytes met `image/png` of `image/jpeg`. Ontbrekend: 404.

Niet ingelogd GET `/` en `/api-docs.html`. API: 401.

CUSTOMER Catalogus, tafels, afbeeldingen lezen; eigen reserveringen.

STAFF Alle API-endpoints.

Onbekende rol Geen API-toegang.

JWT's moeten geldig ondertekend zijn, de ingestelde issuer hebben en `restaurant-api` in `aud` bevatten.

Bronnen: mijn ideefase, de eindopdracht, de Spring Security JWT-documentatie en de Keycloak-import. OpenAI Codex heeft meegeholpen. De prompts staan in `docs/ai-prompts.md`.